

GABARITO DOS EXERCÍCIOS - AULA 22 - DESENVOLVIMENTO DE SISTEMAS

Exercício 1 — Prometendo o Futuro

A criação de uma Promise básica requer a passagem de uma função de *callback* com o parâmetro `resolve` (e opcionalmente `reject`). Utilizamos o `.then()` para capturar o resultado de sucesso.

JavaScript

```
function checarSenha(senha) {
  return new Promise((resolve) => {
    setTimeout(() => {
      if (senha === "senai123") {
        resolve("Acesso Permitido");
      }
    }, 1000);
  });
}
```

```
// Invocando com o .then()
checarSenha("senai123").then(resultado => {
  console.log(resultado);
});
```

Exercício 2 — Consumidor Assíncrono

Refatorar para `async/await` torna a leitura do código sequencial e muito mais limpa, eliminando a necessidade de encadear métodos `.then()`.

JavaScript

```
async function tentarAcesso() {
  const resultado = await checarSenha("senai123");
  console.log(resultado);
}
```

```
tentarAcesso();
```

Exercício 3 — Simulando Desastre na Rede

Aqui a Promise ganha o poder de falhar (usando o `reject`). Ao usar `async/await`, o padrão para capturar falhas deixa de ser o `.catch()` e passa a ser o bloco `try/catch` tradicional.

JavaScript

```
function checarSenha(senha) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (senha === "senai123") {
        resolve("Acesso Permitido");
      } else {
        reject("Acesso negado: Credenciais inválidas");
      }
    }, 1000);
  });
}
```

```
async function tentarAcesso() {
```

```
try {
  // Forçando o erro enviando uma senha incorreta
  const resultado = await checarSenha("senha_errada");
  console.log(resultado);
} catch (erro) {
  console.error(erro); // Irá imprimir "Acesso negado: Credenciais inválidas"
}
}
```

```
tentarAcesso();
```

Exercício 4 — Feedback Visual Duplo

Para manipular atrasos sequenciais sem aninhar funções, criamos uma função de espera genérica (`delay`) e pausamos a execução da interface passo a passo.

JavaScript

```
// Função auxiliar para aguardar o tempo
```

```
function delay(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
```

```
async function atualizarStatus() {
```

```
  // Supondo a existência de um elemento <div id="status-texto"></div>
```

```
  const elemento = document.getElementById("status-texto");
```

```
  elemento.innerHTML = "Conectando...";
```

```
  await delay(1500);
```

```
  elemento.innerHTML = "Verificando dados...";
```

```
  await delay(1000);
```

```
  elemento.innerHTML = "Usuário Autenticado";
```

```
}
```

```
atualizarStatus();
```

Exercício 5 — O Guardião do Status

É comum que bibliotecas como o `fetch` não disparem erros sozinhas em requisições HTTP 404 ou 500. Nós precisamos checar o `status` manualmente e forçar a execução para a área de falha utilizando `throw new Error`.

JavaScript

```
// Simulando uma API que sempre "resolve", mas devolve um status de erro lógico
```

```
function simularChamadaApi() {
```

```
  return new Promise(resolve => {
```

```
    setTimeout(() => {
```

```
      resolve({ status: 404, message: "Produto não encontrado" });
```

```
    }, 800);
```

```
  });
```

```
}
```

```

async function verificarProduto() {
  try {
    const resposta = await simularChamadaApi();

    // O Guardião: Checa o status antes de prosseguir
    if (resposta.status !== 200) {
      throw new Error("Produto Inexistente");
    }

    console.log("Produto carregado com sucesso!");
  } catch (erro) {
    console.error(erro.message); // Cairá aqui imprimindo "Produto Inexistente"
  }
}

```

```

verificarProduto();

```

Exercício 6 — O Algoritmo Teimoso (Retry)

O padrão *Retry* pode ser facilmente construído colocando o `try/catch` dentro de uma estrutura de repetição, como um laço `for`.

JavaScript

// Simula uma rede que falha na maioria das vezes

```

function buscarDadosInstaveis() {
  return new Promise((resolve, reject) => {
    const sucesso = Math.random() > 0.7; // Apenas 30% de chance de sucesso
    setTimeout(() => sucesso ? resolve("Dados obtidos!") : reject("Falha na rede"), 500);
  });
}

```

```

async function executarComRetry(tentativasIniciais = 3) {
  for (let tentativaAtual = 1; tentativaAtual <= tentativasIniciais; tentativaAtual++) {
    try {
      console.log(`Tentativa ${tentativaAtual}...`);
      const resultado = await buscarDadosInstaveis();
      console.log("Sucesso:", resultado);
      return; // Sai da função imediatamente após o sucesso
    } catch (erro) {
      if (tentativaAtual === tentativasIniciais) {
        console.error(`Erro definitivo após ${tentativasIniciais} tentativas.`);
      } else {
        console.warn(`A rede falhou na tentativa ${tentativaAtual}. Reconectando...`);
      }
    }
  }
}

```

```

executarComRetry();

```

Exercício 7 — Corrida de Cavalos com Promise.all

Quando as requisições não dependem uma da outra, rodar de forma sequencial é perda de tempo. O `Promise.all` dispara todas simultaneamente e espera apenas o tempo da mais demorada (neste caso, 3 segundos no total, em vez de 5).

JavaScript

```
function buscarVendas() {
  return new Promise(resolve => setTimeout(() => resolve("Vendas do dia: R$ 8.500"), 2000));
}

function buscarMetas() {
  return new Promise(resolve => setTimeout(() => resolve("Meta Mensal: 85% Concluída"), 3000));
}

async function carregarDashboard() {
  console.time("Tempo de Carregamento");

  // Ambas são disparadas ao mesmo tempo
  const [vendas, metas] = await Promise.all([buscarVendas(), buscarMetas()]);

  console.log(`${vendas} | ${metas}`);
  console.timeEnd("Tempo de Carregamento"); // Vai registrar aprox 3000ms
}

carregarDashboard();
```

Exercício 8 — O Fantasma da IA Generativa

Integramos um `setInterval` (para o comportamento contínuo) que é interrompido pelo término do tempo gerado aleatoriamente pela Promise usando `clearInterval()`.

JavaScript

```
async function gerarTextolA() {
  // Supondo a existência de <textarea id="chat-ia"></textarea>
  const textArea = document.getElementById("chat-ia");
  textArea.value = "";

  // Sorteia um tempo entre 3000ms e 8000ms
  const tempoPensamento = Math.floor(Math.random() * (8000 - 3000 + 1)) + 3000;

  // Inicia o efeito visual
  const efeitoPensando = setInterval(() => {
    textArea.value += "Pensando...\n";
  }, 1000);

  // Aguarda o tempo aleatório da "geração"
  await new Promise(resolve => setTimeout(resolve, tempoPensamento));

  // Limpa o loop e finaliza
  clearInterval(efeitoPensando);
  textArea.value += "\n[A resposta completa da IA foi gerada!]";
}

gerarTextolA();
```

Exercício 9 — O Assombro do Timeout

A chave para um temporizador máximo é o `Promise.race()`. Ele aceita um Array de Promises e retorna o resultado (sucesso ou falha) daquela que terminar **primeiro**.

JavaScript

```
function fetchComTimeout(promessaOriginal, limiteMs) {
  // Cria uma promessa que existe apenas para rejeitar após X milissegundos
  const promessaTimeout = new Promise( (_, reject) => {
    setTimeout(() => {
      reject(new Error("Erro de Timeout: Tempo limite excedido"));
    }, limiteMs);
  });

  // A primeira que terminar ganha a corrida
  return Promise.race([promessaOriginal, promessaTimeout]);
}

// Simulando uso
const chamadaLenta = new Promise(resolve => setTimeout(() => resolve("Sucesso da API!"), 5000));

async function testarResultados() {
  try {
    // Definimos limite de 3000ms. Como a chamada leva 5000ms, o Timeout vencerá.
    const resultado = await fetchComTimeout(chamadaLenta, 3000);
    console.log(resultado);
  } catch (erro) {
    console.error(erro.message);
  }
}

testarResultados();
```

Exercício 10 — O Exorcista do Callback Hell

A maior vantagem do `async/await` é tornar dependências sequenciais elegantes de ler. Cada etapa só inicia quando a de cima termina e entrega a variável, matando a clássica "pirâmide do terror" da indentação.

JavaScript

// 1. Funções isoladas retornando Promises limpas

```
function lerIdUsuario() {
  return new Promise(resolve => setTimeout(() => resolve(105), 1000));
}

function buscarPrivilegios(id) {
  return new Promise(resolve => setTimeout(() => resolve(["admin", "financeiro"]), 1500));
}

function liberarMenus(privilegios) {
  return new Promise(resolve => setTimeout(() => {
    if (privilegios.includes("admin")) {
```

```
    resolve("Dashboard Completo Liberado");  
  }  
}, 1000));  
}
```

// 2. Orquestrador sequencial livre de "Callback Hell"

```
async function inicializarSistema() {  
  try {  
    console.log("Lendo ID...");  
    const id = await lerIdUsuario();  
  
    console.log(`Buscando permissões do usuário ${id}...`);  
    const privilegios = await buscarPrivilegios(id);  
  
    console.log("Montando interfaces...");  
    const menus = await liberarMenus(privilegios);  
  
    console.log("Sistema Pronto:", menus);  
  } catch (erro) {  
    console.error("Falha na inicialização do sistema:", erro);  
  }  
}
```

```
inicializarSistema();
```